The software factory is already running.

Inside the workflows we use to build Paradev.

01

Customers talk to agents

02

Agents hand work to agents

03

People inspect the evidence

The unit of work is becoming a whole feedback loop.

The interesting change is between the prompts.

A coding assistant helps with a task. An operating workflow carries it across tools and people.

Signal

A customer report, review request, alert or schedule

Execution

A workspace with code, tools and an agent

Evidence

A PR, a report and an explicit human handoff

Each handoff needs context, ownership and an observable result.

Our reviewer gets a computer, too.

The Home is a configured cloud workflow with instructions to inspect behavior at the PR head.

Trigger

Assignment or review request to superset-home; selected authors

Inspect

Read the diff, surrounding code, discussion and CI

Exercise

Run changed surfaces and capture evidence, or name the gap

The review can include running the app, not just reading the patch.

A verdict comes with something you can inspect.

One report connects the change, the evidence and the workspace where it was checked.

Report

Summary, findings, evidence, questions and test notes

Delivery

A GitHub comment and a reply in the PR’s Slack thread

Lifecycle

Sleep after the report; cleanup on PR merge or close

A person can follow the links and continue the investigation.

Useful review lives in the interactions.

Two recent findings show what a second agent can contribute.

PR #9678

A reviewer filter worked alone, but combined with another filter it widened the results.

PR #9677

An auth fix missed adjacent interactive subcommands and a similar OpenCode path.

These are concrete findings; they are not a measured reviewer accuracy rate.

The customer's thread becomes the work queue.

Martin reports a photo upload that hangs across Wi-Fi and mobile data.

Recognize

Coda matches the report to SUPER-3442 instead of filing another copy.

Investigate

Avi asks for a deeper debug; Coda opens a cloud session.

Return

Coda posts the root cause, PR and evidence back to the thread.

Customer → Coda → coding session → The Home → Harshith.

It looked like a network bug. It was a state-machine bug.

The attachment entered an “uploading” state that the host path never settled.

Before

Adding a photo seeded progress = 0. The host uploads at send time, so Send stayed disabled.

Proposed fix

Create upload state when an upload actually begins. Adding a photo only adds the attachment.

PR #9675 was open and unmerged when checked.

The reviewer said what it could not verify.

The Home checked tests and the cloud attachment path, then named the missing host environment.

Checked

The review reports a failing-on-base regression test and an unchanged cloud path in the simulator.

Still missing

The host-workspace composer could not be checked in the app: no live relay or host-service.

A review can be useful while the affected end-to-end path remains unverified.

Coda can carry the thread. People still set the boundary.

A tooltip bug expanded into a request for multiple PRs and merge progress.

Customer asks

Remove the overlapping tooltip and show all session PRs.

Human redirects

Satya separates the multi-PR feature into a high-priority ticket.

Agent responds

Coda files SUPER-3380; PR #9594 keeps the narrower fix.

Keeping work small is part of operating the loop.

Events can start real workspaces.

Automations connect an incoming event to an agent, context and somewhere to run.

Inputs

GitHub, Slack, Linear, Sentry, Notion, Teams, schedules and raw webhooks

Routing

Choose a project, local device or cloud environment, and an agent

Continuity

Start fresh, reuse a workspace, or continue the prior conversation

The output is a workspace a teammate can open and continue.

Anything that can POST can call in an agent.

A deploy script, CI job or internal tool can pass structured context into an automation.

Send

POST JSON with a bearer token to the automation’s webhook URL

Receive

The authenticated payload enters the automation event pipeline

Act

An agent receives the event details before its task prompt

Example to build: failed deploy → investigate → evidence + proposed fix.

The system has a loop for its own failures.

A scheduled agent is instructed to turn recurring Dev failures into durable fixes.

Observe

Gather failed turns, retries, lost work and production evidence

Understand

Group by cause, check existing tickets, keep a shared notebook

Encode

A repeated repair becomes a reconciler rule or a code fix

The instructions say: the same fix a third time belongs in code.

Three kinds of improvement. Three different claims.

The evidence is strongest for faster feedback and explicit operational learning.

Product

Reports become proposed fixes, with evidence and review.

Process

Recurring failures can change code, prompts and recovery rules.

Model

These workflows provide no evidence of model-weight improvement.

A useful system can improve its operating process without training a new model.

Count completed outcomes and the cost of getting there.

Instrument the whole loop before claiming a productivity multiplier.

Delivery

Report → proposed fix → reviewed merge → successful use

Quality

Verified findings, rework, reverts and escaped defects

Cost

Human attention, model spend and workspace runtime per outcome

A started workspace is a dispatch result. A working fix is a product result.

Build the feedback loop you can actually inspect.

Paradev already connects customer context, coding sessions, review evidence and human decisions.

Start with a signal

Put work where the context already lives.

Make evidence travel

Return a PR, a report and the limits of the check.

Keep an owner

Someone owns scope, the merge and the result in production.

That is the software factory worth showing.